

Lab 3: Arithmetic Circuits

Prepared by

Student name: Aryan Qureshi

Student ID: 300467938

Student name: Borna Chadegani

Student ID: 300488971

Group # 10

ITI 1100 Digital Systems I

Professor: Dr. Aneta Traikova, PhD, P. Eng.

Presented to:

TA and Lab Supervisor:

Dorsa Khodabakhsh (dkhod005@uottawa.ca)

Rahil Ghalamboland (rghal080@uottawa.ca)

Lauren Hendley (lhend093@uottawa.ca)



uOttawa

School of Electrical Engineering and Computer Science

École de science informatique et de génie électrique

Faculty of Engineering | Faculté de génie

Summary

The lab involved designing and implementing arithmetic circuits using hierarchical design techniques in Quartus. The experiment began with constructing a one-bit full adder and verifying its operation through truth tables and simulation waveforms. The full adder was then used as a building block to create an 8-bit parallel adder capable of performing binary addition.

The parallel adder was modified to function as a two's complement adder/subtractor by introducing XOR gates and a control signal to enable signed subtraction. An overflow detection circuit was also implemented to identify cases where signed arithmetic exceeded the allowable range for 8-bit numbers. Finally, a 4-bit multiplier was designed using partial product generation and cascaded adders.

Introduction

Background

Arithmetic operations form the foundation of digital systems. In digital electronics, arithmetic circuits operate on binary numbers using logic gates to perform operations such as addition, subtraction, and multiplication. The half adder and full adder are fundamental building blocks used to implement binary addition. By combining multiple full adders, larger parallel adders can be constructed to handle multi-bit numbers.

Two's complement representation is commonly used in digital systems to represent signed numbers. By modifying a parallel adder with XOR gates and a control signal, the same hardware can perform both addition and subtraction. Overflow detection logic is required to ensure that signed results remain within the permissible range for a given bit width.

Multiplication in digital systems is achieved through partial product generation and repeated addition. By cascading adders and logic gates, binary multipliers can be constructed to perform multiplication of multi-bit operands.

These arithmetic circuits demonstrate how complex mathematical operations can be implemented using basic logic gates and hierarchical design techniques.

Objective of Report

The primary objective of this report is to design, simulate, and implement arithmetic circuits using hierarchical methods in Quartus. The specific goals of this experiment include:

- Designing and verifying a one-bit full adder using Boolean expressions and truth tables
- Expanding the full adder into an 8-bit parallel ripple carry adder
- Modifying the 8-bit adder to perform two's complement subtraction using a control signal
- Implementing an overflow detection circuit for signed arithmetic
- Designing and simulating a 4-bit binary multiplier using partial product generation
- Verifying the correctness of each circuit through simulation waveforms
- Validating the circuit behavior through hardware testing on the Altera board

Equipment

Hardware

- **Altera DE2-115 Development and Education Board**
 - Used to implement and test arithmetic circuits (full adder, 8-bit adder/subtractor, multiplier)
 - Toggle switches used as binary input operands (A, B, SUB, C_IN0)
 - LEDs used to display Sum, Carry, and Overflow outputs

USB-Blaster Programming Cable

Used to program the FPGA device on the DE2-115 board

Software:

- **Intel Quartus Prime / Quartus II Graphic Editor**
 - Used to create hierarchical schematic designs
 - Used to build full adder blocks
 - Used to generate symbols for reuse in higher-level designs
 - Used to compile and simulate arithmetic circuits
 - Used to assign FPGA pins for hardware implementation

Quartus Simulation Waveform Editor (.vwf files)

- Used to verify functional correctness of arithmetic circuits
- Used to compare theoretical results with simulated outputs

Logic Components:

- XOR gates
- AND gates
- OR gates
- NAND gates (for alternative implementations)
- NOR gates (for alternative implementations)
- Full Adder hierarchical blocks
- Parallel adder blocks
- Overflow detection logic

Input and Output Devices:

- Toggle switches on DE2-115 board for:

- Operand A (8-bit input)
- Operand B (8-bit input)
- SUB control signal
- C_IN0 (Carry input)

- LEDs on DE2-115 board for:

- 8-bit Sum output
- Carry Out
- Overflow indicator

Additional Equipment

- Laboratory computer capable of running Quartus
- USB cable for board connection

Pre-Lab

The objective of the pre-lab was to analyze the arithmetic circuits required for this experiment and verify their theoretical operation before implementing them in Quartus. The focus was on understanding the behavior of the full adder, multi-bit addition, and binary multiplication through manual calculations and truth tables.

Part 1 - One-Bit Full Adder

The Boolean expressions for the full adder were derived and verified:

$$\text{SUM} = A \oplus B \oplus C_{\text{IN}}$$

$$C_{\text{OUT}} = AB + AC_{\text{IN}} + BC_{\text{IN}}$$

A complete truth table was constructed to confirm the correctness of the SUM and C_OUT outputs for all possible input combinations.

1. prelab part 1:

$$C_{\text{out}} = AB + (C_{\text{in}}(A \oplus B))$$
$$\text{Sum} = (A \oplus B) \oplus C_{\text{in}} \text{ or } A \oplus B \oplus C$$

A	B	C _{in}	C _{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

The results confirmed that the SUM output follows XOR behavior and the carry output is asserted whenever at least two inputs are high.

Part 2 - 8-Bit Addition

Manual binary addition calculations were performed using the input values provided in the lab instructions. The expected binary and hexadecimal results were calculated in advance to compare with simulation outputs later in the lab.

Part 2:		Binary Input	Carry	Sum	Hex
1.	$\begin{array}{r} 01111111 \\ + 00000001 \\ \hline 10000000 \end{array}$	$(0)_2$	$(1000\ 0000)_2$	$(7F)_{16} + (1)_{16} = (80)_{16}$	
2.	$\begin{array}{r} 11111111 \\ + 00000001 \\ \hline 10000000 \end{array}$	$(1)_2$	$(0000\ 0000)_2$	$(FF)_{16} + (01)_{16} = (100)_{16}$ $= (100)_{16}$ $= (C0)_{16} + (40)_{16}$ $= (100)_{16}$	
3.	$\begin{array}{r} 11000000 \\ + 01000000 \\ \hline 10000000 \end{array}$	$(0)_2$	$(0000\ 0000)_2$	$(C0)_{16} + (80)_{16} = (200)_{16}$ $= (200)_{16}$	
4.	$\begin{array}{r} 11000000 \\ + 10000000 \\ \hline 10000000 \end{array}$				

These calculations established the expected results for verifying the 8-bit ripple carry adder.

Part 3 - Two's Complement Signed Arithmetic

Signed 8-bit binary arithmetic was performed manually using two's complement representation. Each pair of inputs was added or subtracted, and the expected binary sum, carry, overflow, hexadecimal equivalent, and signed decimal value were calculated.

Overflow was determined using the rule:

$$\text{Overflow} = \text{Carry_in_MSB} \oplus \text{Carry_out_MSB}$$

Qr. #	Part 3:	Binary inputs	overflow	carry	Binary sum	Hex	2's (Decimal)
1)		0111 111 + 0000 0001	1	0	10000000	80	-128
2)		1111 111 + 0000 0001	0	1	0000 0000	00	0
3)		0000 0000 - 0000 0001	0	0	1111 1111	FF	-1
4)		0000 0000 - 0111 1111	0	0	10000001	81	-127
5)		11000 000 + 0100 0000	0	1	0000 0000	00	0
6)		11000 000 + 1000 0000	1	1	0100 0000	40	64

The results confirmed that overflow occurs only when the signed result exceeds the allowable range for 8-bit two's complement numbers. These calculated values were later used to verify the correctness of the simulated adder/subtractor circuit.

Part 4 - 4-Bit Multiplication

Binary multiplication was performed manually using partial product generation. Each partial product was calculated and summed to determine the expected final product.

Part 1: #4

- Both NAND and NOR require 9 gates, the NAND implementation is better in terms of total circuit cost and complexity. NAND has fewer logic levels resulting in lower delay and improved performance. It also requires slightly simpler wiring.

Part 4: Table 4

Grid	A x B	Binary Output	Hex Equivalent
1	1010 * 1000	01010000	50
2	0000 * 1111	00000000	00
3	0110 * 1110	01010100	54
4	1111 * 1111	11100001	E1

The manually calculated results were later used to verify the correctness of the multiplier circuit implemented in Quartus.

Design Procedure

The purpose of this experiment was to design and simulate arithmetic circuits beginning with a 1-bit full adder, then expanding into an 8-bit parallel adder, followed by a two's complement adder/subtractor with overflow detection, and finally implementing a 4-bit multiplier using design techniques

Part 1 - Full Bit Adder

The first stage of the lab involved creating a one-bit full adder. The full adder consists of three inputs:

- A
- B
- C_IN

And two outputs:

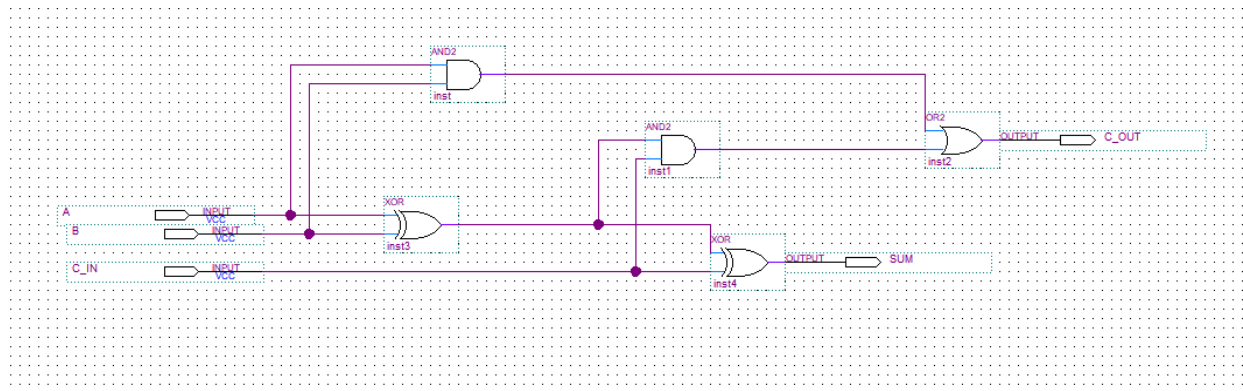
- SUM
- C_OUT

The Boolean expressions used were:

$$\text{SUM} = A \oplus B \oplus \text{C_IN}$$

$$\text{C_OUT} = AB + AC_IN + BC_IN$$

A new Quartus project was created. A Block Diagram was added to the project. XOR, AND, and OR gates were inserted and wired according to the full adder equations. The circuit was compiled to verify there were no errors.

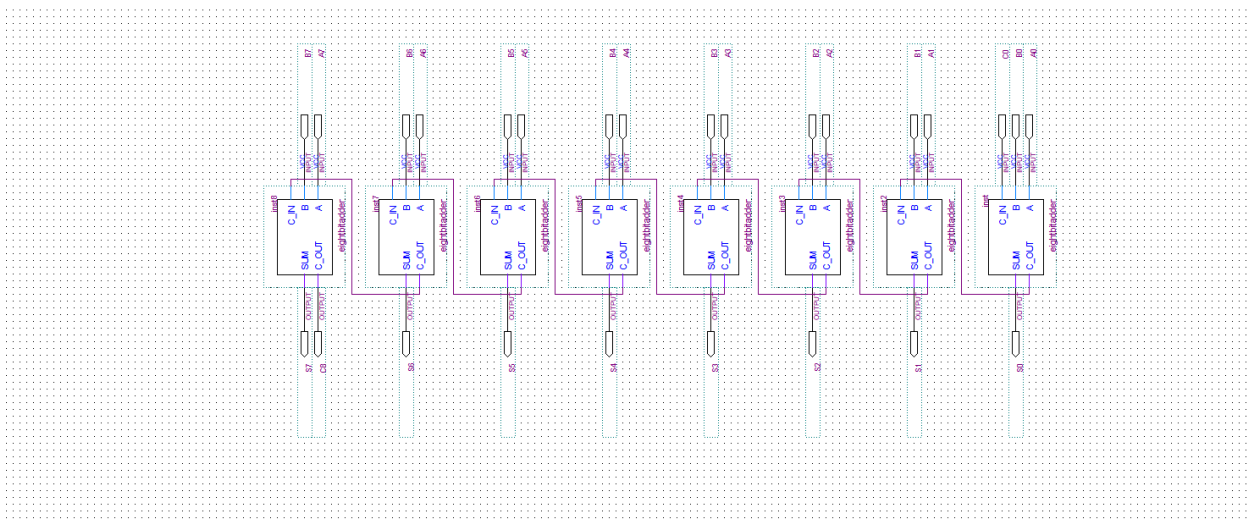


After compilation, a waveform simulation was created. All eight possible input combinations were applied to verify the correctness of SUM and C_OUT. The simulation results matched the theoretical truth table exactly.

Part 2 - 8 Bit Parallel Adder

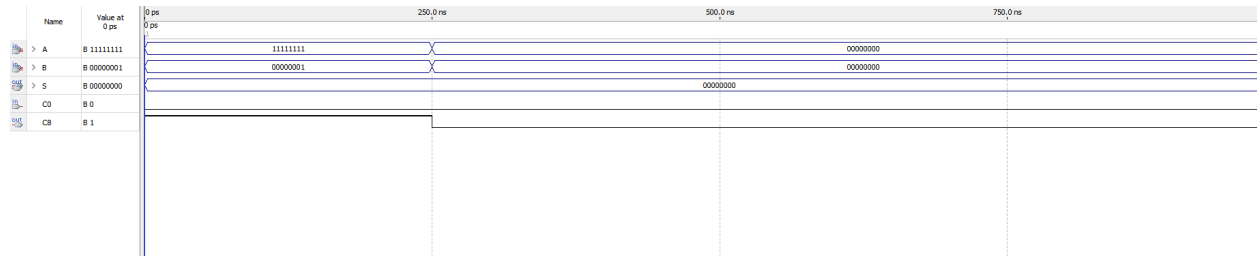
Using the previously created full adder block, an 8-bit ripple carry adder was constructed.

Eight instances of the full adder symbol were inserted into a new block diagram sheet. The carry-out of each stage was connected to the carry-in of the next stage, forming a ripple carry structure. Inputs A and B were connected to each corresponding full adder stage. The first stage carry input was set to C_IN.



A simulation was then performed using the binary values provided in Table 2 of the lab instructions. The resulting binary sums and carry outputs were compared to the expected theoretical results in both binary and hexadecimal form. The simulation outputs matched the calculated results.





After simulation verification, input pins were assigned to toggle switches on the DE2-115 board, and output pins were assigned to LEDs for observing the sum and carry output.

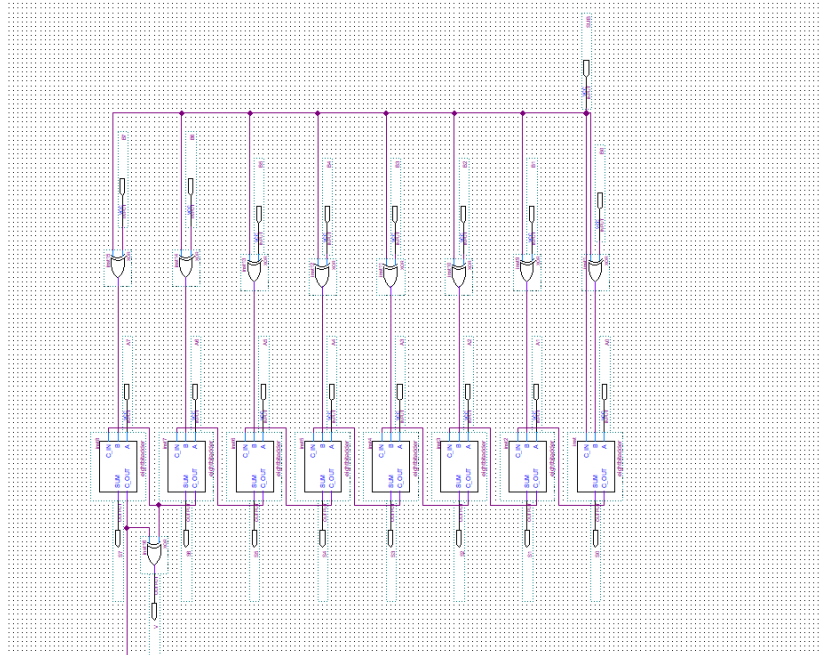
Part 3- Two's Complement Adder/Subtractor

The 8-bit adder was then modified to perform subtraction using two's complement arithmetic.

To achieve this, XOR gates were inserted before each B input bit. A control signal labeled SUB was connected to all XOR gates. When $SUB = 1$, the B inputs are inverted, producing the one's complement of B. The SUB signal was also connected to the initial carry input (C_IN), thereby adding 1 when performing subtraction.

Operation summary:

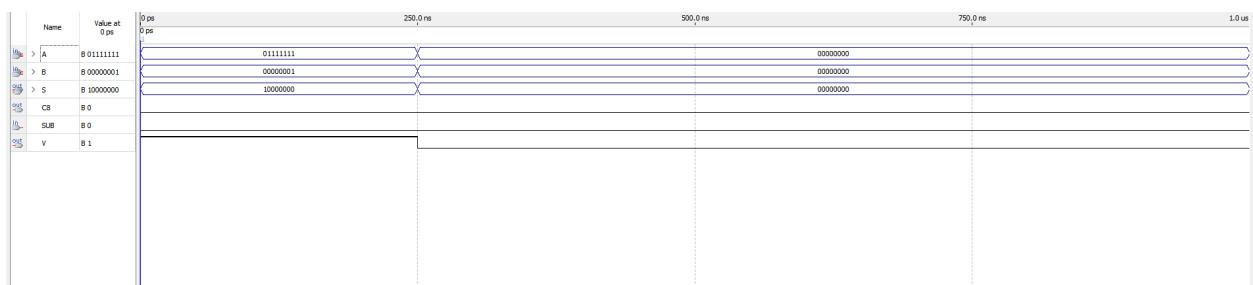
- $SUB = 0 \rightarrow A + B$
- $SUB = 1 \rightarrow A + B' + 1 \text{ (} A - B \text{)}$

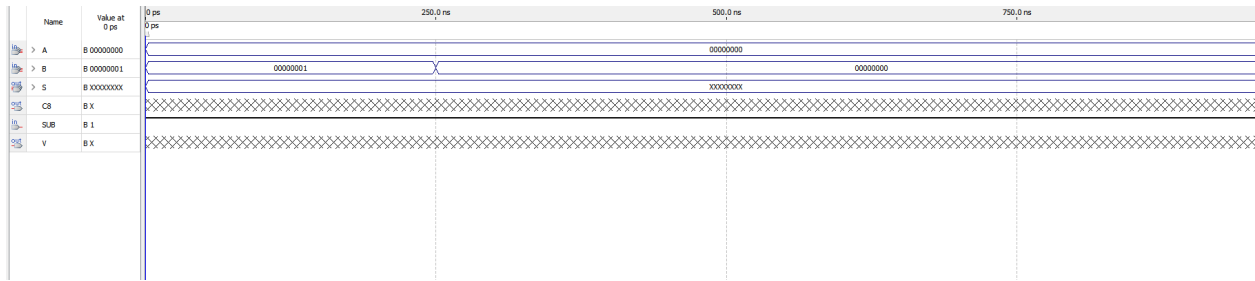


To detect signed overflow, an overflow detection circuit was added. Overflow was implemented by comparing the carry into the most significant bit (MSB) and the carry out of the MSB:

$$\text{Overflow} = \text{Carry_in_MSB} \oplus \text{Carry_out_MSB}$$

The design was compiled and simulated using the signed binary values from Table 3. The waveform showed correct results for addition, subtraction, and overflow cases.

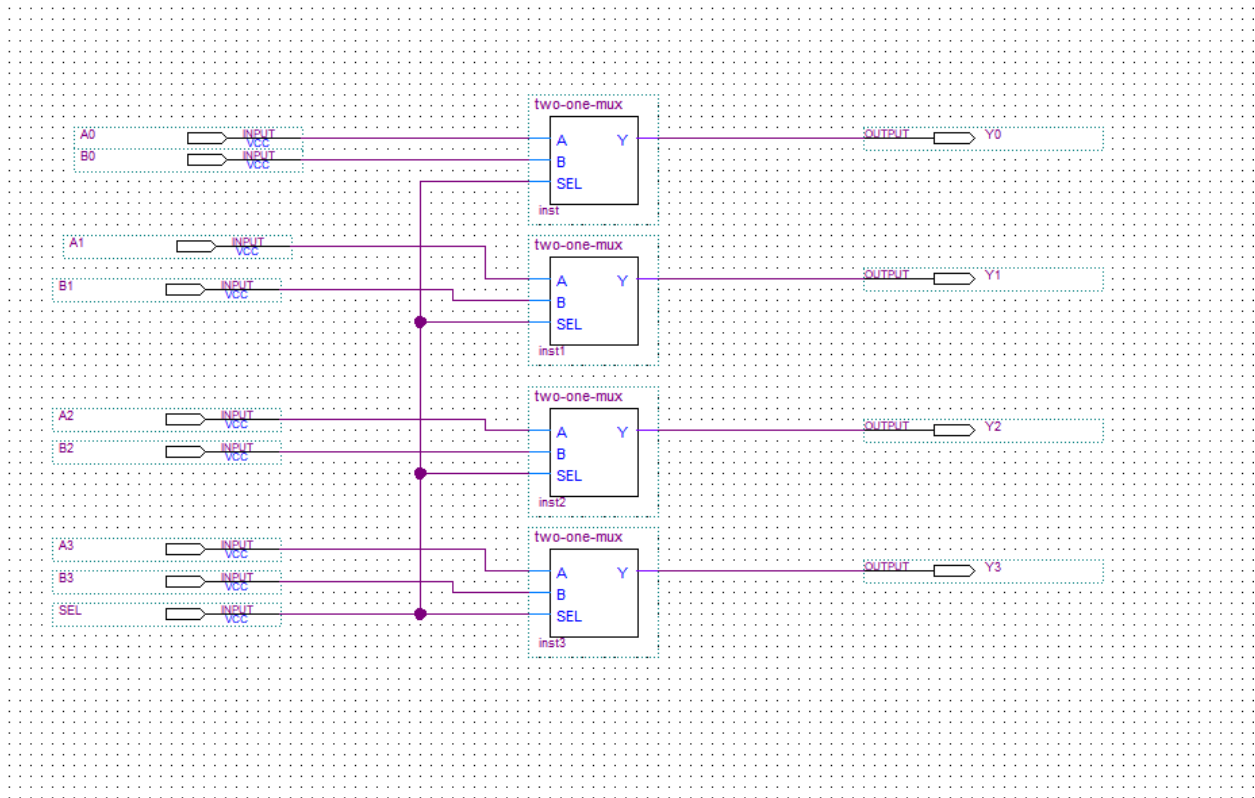




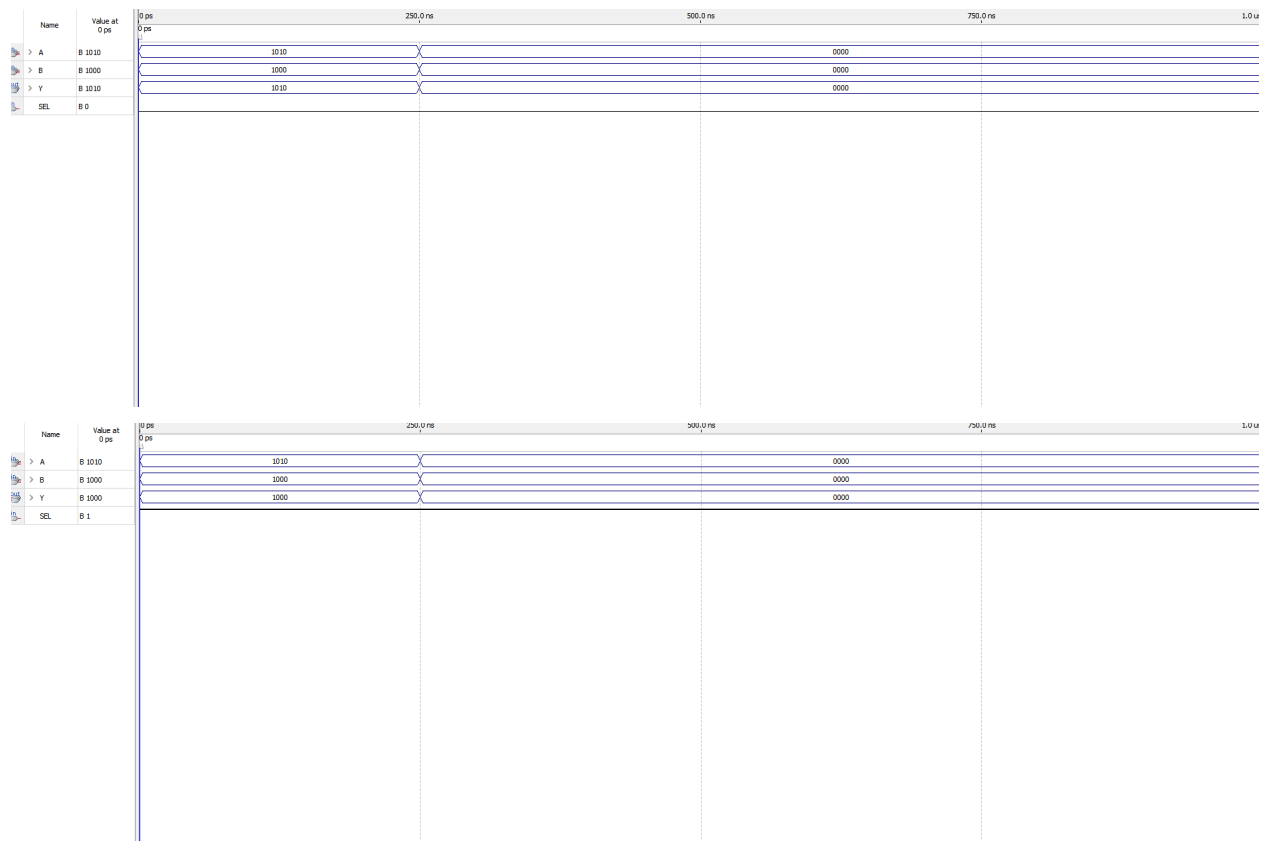
Part 4- 4-Bit Multiplier

The multiplier was implemented using partial product generation and cascaded addition.

First, AND gates were used to generate the partial products of each bit of A with each bit of B. These partial products were then added together using full adder blocks to produce the final 8-bit product output.



After compiling the project, simulations were performed using the binary values provided in Table 4.



The circuit was then assigned to toggle switches and LEDs on the DE2-115 board and tested for correct operation.

Discussion of Results

The experimental results obtained from simulation and hardware implementation confirmed the theoretical behavior of the arithmetic circuits designed in this lab. Each stage of the experiment demonstrated correct logical and arithmetic functionality when compared to the expected theoretical results derived during the pre-lab.

For the one-bit full adder, the waveform simulation verified that the SUM and C_OUT outputs matched the truth table for all eight possible combinations of A, B, and C_IN. The SUM output correctly followed the three-input XOR behavior, while the carry output was asserted only when at least two of the inputs were high. There was no inconsistency between the theoretical Boolean expressions and the simulated outputs, confirming the correctness of the full adder implementation.

The 8-bit parallel adder demonstrated proper ripple carry operation. The carry-out from each stage propagated sequentially into the next stage, producing correct 8-bit sum results. Simulation

results matched the expected binary and hexadecimal values provided in Table 2. Although ripple carry adders introduce propagation delay due to carry transfer, the functional simulation confirmed that the arithmetic results were correct for all tested input combinations.

When the adder was modified into a two's complement adder/subtractor, the circuit successfully performed both addition and subtraction using the SUB signal. When SUB was low, the circuit behaved as a normal adder. When SUB was high, the B inputs were inverted and 1 was added through the initial carry input, effectively computing $A - B$ using two's complement arithmetic. The waveform simulation confirmed that both signed addition and subtraction cases produced correct results.

The overflow detection logic also operated correctly. Overflow occurred only when the carry into the most significant bit differed from the carry out of the most significant bit. In all test cases where signed overflow was expected, the overflow output was asserted correctly. No false overflow signals were observed in valid cases.

The 4-bit multiplier successfully generated correct product outputs for all test cases provided in Table 4. The partial products were correctly generated using AND gates and summed using cascaded adders. The final 8-bit product matched the theoretical binary multiplication results. Both simulation and hardware testing confirmed the reliability of the design.

Overall, there was strong agreement between theoretical calculations and simulation waveforms.. No unexpected behavior was observed. The hierarchical design approach improved organization, particularly when using the full adder as a reusable building block for larger circuits.

Conclusion

This lab focused on the design and implementation of arithmetic circuits using hierarchical methods in Quartus. Starting with a one-bit full adder, the circuit was expanded into an 8-bit parallel adder and later modified into a two's complement adder/subtractor with overflow detection. A 4-bit multiplier was also designed using partial product generation and cascaded addition.

Simulation results confirmed that all circuits produced outputs that matched the expected theoretical calculations for addition, subtraction, signed arithmetic, and multiplication. The overflow detection logic correctly identified cases where signed results exceeded the allowable range.

Both simulation and hardware testing on the DE2-115 board showed consistent and accurate results, demonstrating the correctness of the designs. Overall, the lab reinforced the importance of hierarchical design and confirmed the reliability of arithmetic circuit implementation using Quartus and hardware.

Appendices

Appendix A – One-Bit Full Adder (Part 1)

- part1.bdf
- part1.bsf
- part1.qpf
- part1.qsf
- part1.qws
- Waveform.vwf

Appendix B – 8-Bit Parallel Adder (Part 2)

- eightbitadder.bdf
- eightbitadder.bsf
- eightbitadder.qpf
- eightbitadder.qsf
- eightbitadder.qws
- Waveform.vwf

Appendix C – Two's Complement Adder/Subtractor (Part 3)

- twoscompliment.bdf
- twoscompliment.qpf
- twoscompliment.qsf
- twoscompliment.qws
- Twoscomplment.vwf

Appendix D – 4-Bit Multiplier (Part 4)

- fourbitmultiplexer.bdf
- two-one-mux.bdf
- two-one-mux.bsf
- Multiplexer.qpf
- Multiplexer.qsf
- Multiplexer.qws
- Waveform.vwf

Appendix E – Simulation and Software Environment

- Quartus Simulation Waveform Editor (.vwf files)
- Quartus Block Diagram/Schematic Editor
- Intel Quartus Prime
- Altera DE2-115 FPGA Board